Generation in three parts: the LLM calls

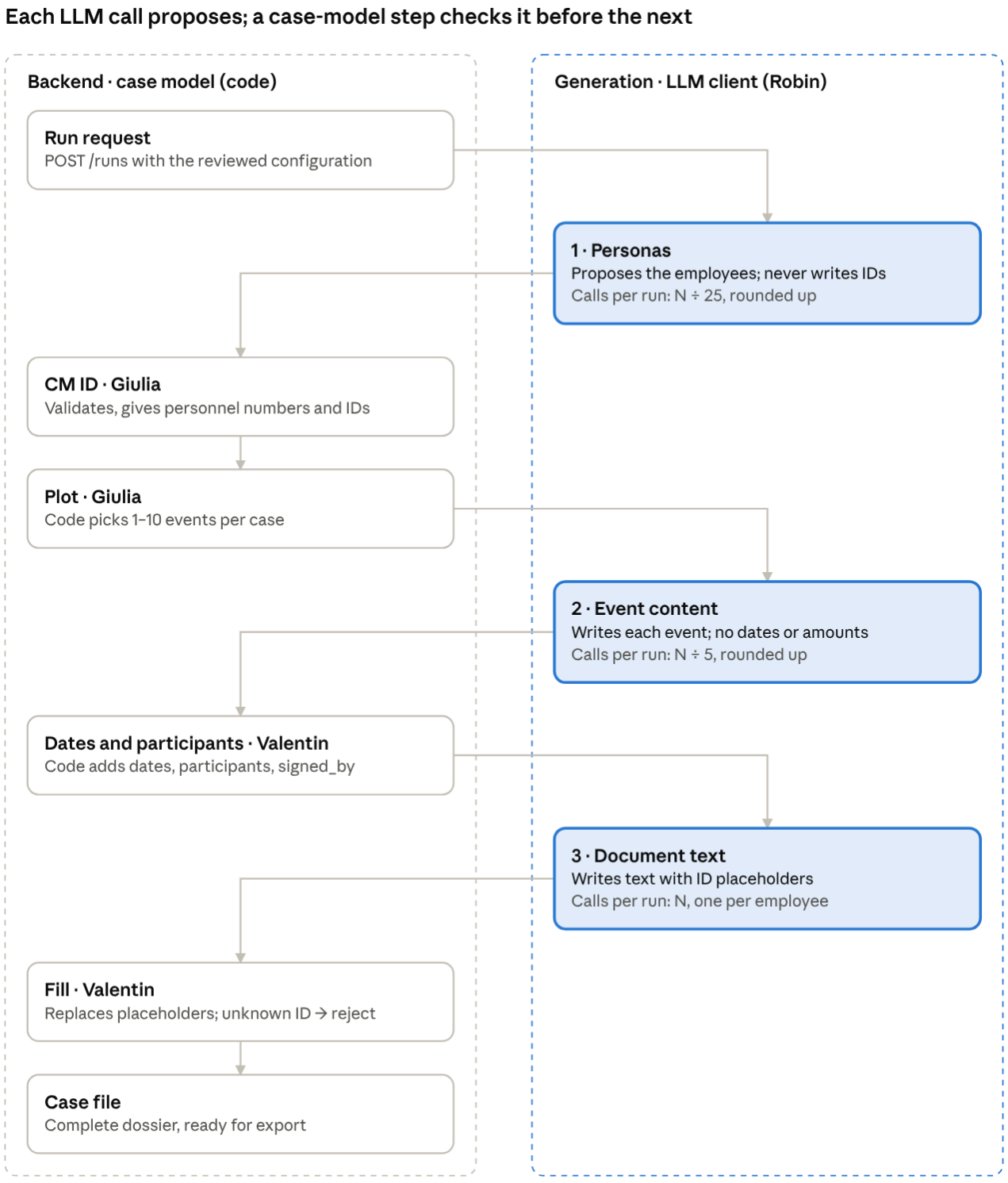
Oct 31, 2026 · @Robin

Summary

Generation splits into three LLM calls that run in a fixed order: **Personas**, **Event content** and **Document text**. Each call only proposes JSON. After each one, a case-model step checks it, gives IDs and saves it, and the next prompt is built from that saved state.

For a run of N employees this is N ÷ 25 + N ÷ 5 + N calls, each rounded up: 13 calls for 10 employees, at most 26 with one retry each.

This follows the 30 October meeting: generation and the case model stay separate, joined by a contract; code picks 1–10 events and the LLM only writes them; the LLM writes documents with ID placeholders that the case model fills; one Sonnet key for the project.



generation in three parts · 3 LLM calls, each followed by a case-model step

The highlighted boxes are the three LLM calls. Every arrow back into the left column is a gate where code checks the answer. N is the number of employees in the run.

Part 1 · Personas

Part 1 proposes the company's employees from the reviewed configuration, in one call per 25 employees. It never writes IDs: CM ID numbers the employees afterwards.

- **Gets:** the reviewed configuration from `POST /runs`: company, departments with positions and salary ranges, number of employees, temporary and part-time shares, as-of date, notes.
- **LLM writes:** per employee, first and last name, department, position, manager or not, hire date, contract type, hours per week, start salary.
- **Returns:** `EmployeesIn`, the shape of `input.employees.schema.json`.
- **Calls:** N ÷ 25, rounded up (the demo's current limit per call).
- **Checked by:** `complete_as` (shape and types, one retry), then CM ID (closed lists, salary ranges, corrections).
- **Next step:** CM ID (Giulia) gives personnel numbers and the `P` and `ORG` IDs.

The demo's prompt (`personasPrompt` in `prompts.ts`) carries over almost as is. One behaviour changes: the demo quietly filled missing employees with offline ones; now a short or malformed answer is retried once, then the run stops with the field named.

Part 2 · Event content

Part 2 writes the content of the events Plot already picked, in one call per 5 employees. It writes no dates, amounts, participants or IDs; code adds those in the next step.

- **Gets:** per employee, the facts CM ID saved (name, position, department, hire date, contract) and Plot's outline: 1–10 event slots, each with a type and a slot key from code.
- **LLM writes:** per slot, the text and choices in the table below, plus a one-line `summary` (the case already has that field).
- **Returns:** `EventContentIn`: per employee number, a list of slot key plus content. This is a new, smaller model than today's `storylines.json`, which still has the LLM writing dates.
- **Calls:** N ÷ 5, rounded up (the demo's storyline batch size).
- **Checked by:** `complete_as`: every slot answered exactly once, values from the closed lists, one retry.
- **Next step:** Dates and participants (Valentin) adds dates within the rules, participants and `signed_by`, then calls `ingest_storylines`.

Event type	LLM writes	Code decides
contract_change	Reason; the new position on a promotion, from the department's list	Date, new salary, contract type, hours
training	Course, duration in days	Date, training provider
appraisal	Rating (closed list), comment	Year, date (30 June), signer
termination	Reason (closed list)	Date (month end, at least 6 months after hire), notice date
absence	Nothing: dates only, never a cause	Start and end date
medical_check	Nothing	Date (every 2 years), result
hire	Nothing	Everything (CM ID adds it)

The demo's `storylinePrompt` already hands the model some of Plot's choices (who leaves, how many sick-leave periods, which appraisal years). The new prompt goes one step further: the event list is fixed, and the model only fills the slots.

Part 3 · Document text

Part 3 writes the free text of each dossier's documents, in one call per employee, using ID placeholders instead of names, dates or amounts. Fill then puts in the facts and rejects any ID the case does not hold.

- **Gets:** the complete case (IDs, dated events, participants, `signed_by`) as the list of placeholders it may use, plus a document plan from code: which documents, for which event, which sections.
- **LLM writes:** the body text of each planned document, for example: *On {{events.EVT-004.date}}, {{personas.P002.name}} reviewed the work of {{personas.P001.name}}.*
- **Returns:** `DocumentTextsIn`: per document key from code, the text with placeholders.
- **Calls:** N, one per employee. All of a dossier's texts are written together, so one manager sounds the same across years.
- **Checked by:** `complete_as`, with the placeholder check inside the model. An unknown ID or field fails validation, and so does a name, date or amount typed out instead of a placeholder. That way it gets the one retry with the bad placeholder named, then the run fails.
- **Next step:** Fill (Valentin) replaces the placeholders and saves the result in `Document.text`; then export (Stylianos).

Proposal for which documents get LLM text:

Document type	Text from
Performance appraisal	LLM: feedback and summary
Termination letter	LLM: letter body
Contract amendment	LLM: cover letter
Employment contract	Template: fixed legal wording
Training certificate	Template
Payslip	Template and rules
Annual income statement	Template and rules
Medical check	Template: no health details from the LLM
Absence report	Template: dates only, never a cause

Payslips are by far the most numerous documents (one per month), so keeping them templated keeps Part 3 at one call per employee.

All LLM calls in one run

A run of N employees makes N ÷ 25 + N ÷ 5 + N generation calls, each rounded up, plus one or two frontend calls. Every call gets at most one retry.

#	Call	Part, owner	Calls per run	Returns	Code step after it
1	Create analysis	Frontend, Daniel	1 per input, again after each edit	The analysis (BE-07)	Known-template check
2	Rule configuration	Frontend, Daniel	0 or 1: only for an unknown template	Template rules	Control layout view
3	Personas	Generation part 1, Robin	N ÷ 25, rounded up	`EmployeesIn`	CM ID
4	Event content	Generation part 2, Robin	N ÷ 5, rounded up	`EventContentIn`	Dates and participants
5	Document text	Generation part 3, Robin	N	`DocumentTextsIn`	Fill

For 10 employees that is 1 + 2 + 10 = 13 generation calls, at most 26 with retries, plus 1–2 frontend calls. The demo caps answers at 300 + 120 tokens per employee for personas and 800 + 2,000 per employee for a storyline batch; document text is not measured yet. Mike's monitoring document turns these into daily calls and tokens for the API request.

The contract between generation and the backend

Generation is three Python functions that the backend's orchestrator calls in order, inside the one service (BE-01). Each returns a validated Pydantic model, so the backend never sees raw model text.

```
propose_personas(config, llm) -> EmployeesIn
write_event_content(cases, outlines, llm) -> EventContentIn
write_document_text(case, document_plan, llm) -> DocumentTextsIn
```

Rules that hold at every boundary:

1. Generation never writes an ID. It may only echo a key that code gave it: an employee number, a slot key, a document key.
2. Generation never writes dates, amounts or participants. Code adds them after Part 2.
3. Each prompt is built from what the case model saved, not from the previous answer.
4. A bad answer gets one retry, then an `LLMError` that names the field. Nothing is half-written.
5. Every call goes through the one LLM client, which records model, tokens, time and retries for Mike's trace (BE-19).

Rule 5 needs one change to the client: `complete()` returns only text, so it should also return the token usage. Each function can be tested offline with `FakeLLM`.

`EXAMPLE_PAYLOAD.md` shows the first boundary (Personas to CM ID) with real data. Parts 2 and 3 still need a worked example.

Open questions

- [ ] Placeholder syntax: `{{personas.P001.name}}` (Giulia's proposal) or `employee_001` (the meeting notes)? Valentin, Robin (AX-021)
- [ ] Which document types get LLM text: the three proposed above, or more? Robin, Stylianos
- [ ] Do start salary and hire date move to code too, like the event dates? Robin, Giulia
- [ ] Does the LLM pick the closed-list values (rating, termination reason), or does Plot? Robin, Giulia

- [ ] Keep the demo's batch sizes (25 and 5), or measure first with the school key? Robin, Mike
- [ ] A second human review after Fill: mentioned on 30 October, not decided. Daniel